

Disentangling Semantic Knowledge and Memorisation in LLM-Based Data Cleaning: A Benchmark on Synthetic vs. Real Tabular Data



Under the Supervision of:
Dr. Faiyaz Ahmad
Associate Professor

Department of Computer Engg.,
Faculty of Engg. & Technology,
Jamia Millia Islamia, New Delhi-110025

Presented By:
Al Ayaan Ansari (23BCS034)
Mohd Adnan Khan (23BCS042)

9th October 2026



Contents

01 Introduction

02 Problem Statement

03 Literature Review

04 Dataset

05 Proposed Algorithm

06 Proposed Architecture

07 Performance Evaluation

08 References

Introduction

- Most real tables have gaps. A sensor stops working, a patient skips a test, or someone leaves a form field blank.
- If the gaps are filled badly, every model trained on that data carries the mistake forward.
- Traditional methods (mean, k-NN, MICE, MissForest) fill gaps using patterns in the numbers alone.

Type	What it means	Example
MCAR – Missing Completely At Random	The gap has nothing to do with any value	A lab machine fails on random days
MAR – Missing At Random	The gap depends on other values we can see	Older patients skip a fitness test more often
MNAR – Missing Not At Random	The gap depends on the missing value itself	People with very high income don't report it

Table 1: Types of missing data (Rubin, 1976)

Introduction (Contd.)

- With LLMs, a row is written out as text and the model guesses the missing value, roughly the way a person would.
- Papers report good results for filling gaps [1, 2, 5], building cleaning workflows [3] and other data preparation tasks [6].
- But most of these tests use famous datasets like Iris, Pima and Adult.
- LLMs can recite many of these datasets from memory [4].
- So did the model understand the data, or just remember it?

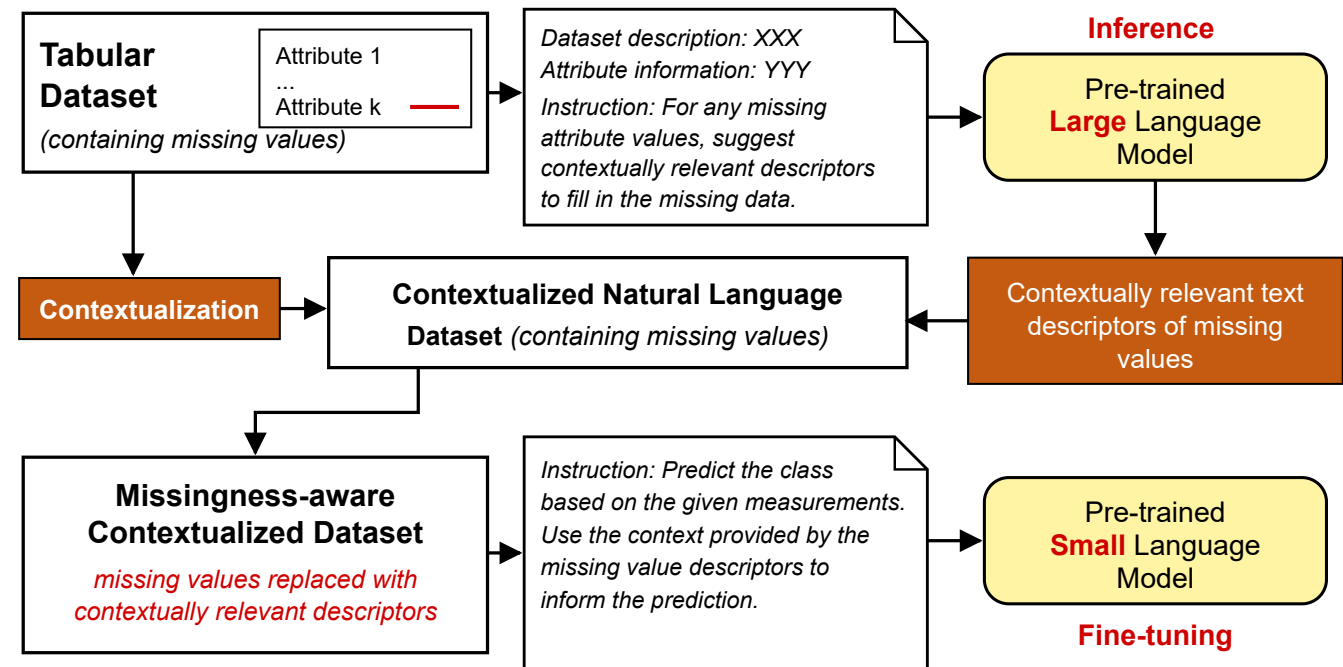


Figure 1: An overview of CRILM, an LLM-based imputation method (redrawn from Hayat and Hasan [1])

Problem Statement

- Papers say LLMs fill missing values better than classical methods.
- But they test on datasets the LLM has probably already seen during training.
- So a good score can mean two very different things: the model understood the data, or it simply remembered the records.
- Until we can tell these apart, we can't trust LLM cleaning on private data like hospital or bank records.
- **Our aim:** build a fair test bench that changes one thing at a time (where the data comes from, and what the columns are called) and see how the LLM's accuracy moves.

Research Questions



No.	Question
RQ1	How much of the LLM's advantage comes from understanding column meanings, and how much from remembering records?
RQ2	If a model has memorised a dataset more, does it do better on it?
RQ3	Does an LLM handle MNAR suspiciously well on famous data, but not on a synthetic copy?
RQ4*	Can a synthetic copy of a dataset rank cleaning methods the same way the real one does?
RQ5*	Do common ways of simulating missing data accidentally give away the hidden values?

Table 2: Research questions * secondary questions

Literature Review



Author & Year	Title	Technique & Model Used	Remark	Dataset	Source of Publication
Hayat & Hasan (2025)	A Context-Aware Approach for Enhancing Data Imputation with Pre-trained Language Models	LLM writes a short note for each missing value; Llama 2 7B is trained on it	Up to 10% better than the best imputer when data is MNAR	6 UCI datasets	COLING 2025
Abate et al. (2026)	Constrained Conditional LLM-based Imputation via Iterative Chained Updates	MICE-style loop where an LLM fills one value at a time (Gemma2-9B)	Small but significant gain over MICE	Pima, DiaBD	Pattern Recognition Letters
Li et al. (2025)	AutoDCWorkflow: LLM-based Data Cleaning Workflow Auto-Generation and Benchmark	LLM agent writes step-by-step cleaning workflows (Llama 3.1 8B, Gemma 2)	Answer F1 went from 0.22 to 0.63	Hospital, Flights and 4 more	Findings of EMNLP 2025

Table 3: Summary of reviewed papers (1 of 2)

Literature Review (Contd.)



Author & Year	Title	Technique & Model Used	Remark	Dataset	Source of Publication
Bordt et al. (2024)	Elephants Never Forget: Memorization and Learning of Tabular Data in Large Language Models	Tests whether a model can recite a dataset row by row (GPT-3.5, GPT-4)	GPT-4 completed 125 of 136 Iris rows from memory	Iris, Wine, Titanic, Adult and newer sets	COLM 2024
He et al. (2025)	LLM-Forest: Ensemble Learning of LLMs with Graph-Augmented Prompts for Data Imputation	Asks GPT-4o several times with similar rows and takes a vote	Best on 8 of 9 datasets; biggest gain under MNAR	9 UCI / Kaggle datasets	Findings of ACL 2025
Zhang et al. (2024)	Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing	Fine-tunes small local LLMs (7B to 13B) for cleaning tasks	13B model slightly beats GPT-4 on average (86.0 vs 84.2)	Adult, Hospital, Buy, Restaurant	EMNLP 2024

Table 4: Summary of reviewed papers (2 of 2)



Literature Review (Contd.)

What the papers got right

- 01** LLMs can fill in missing values and tidy messy tables, often better than older methods.
- 02** You don't need a giant model. 7–9B models running on a normal GPU do well.
- 03** Showing the model a few example rows, or letting it work in steps, helps a lot.
- 04** Column names matter. Hide them and accuracy drops sharply.

What's still missing

- 01** Almost nobody checks whether the model has already seen the test data.
- 02** Good results are credited to the model's "knowledge" without proof.
- 03** Some papers report the biggest LLM gains under MNAR; the most careful study found them inconsistent.
- 04** Tests use small, famous datasets with only a few repeat runs.

Research Gap

- Most papers test LLM cleaners on famous datasets like Iris, Pima and Adult.
- Bordt et al. [4] showed LLMs can recite such datasets from memory, and that this inflates their scores.
- Yet none of the cleaning papers checks for this, hides the column names, or tests on data the model couldn't have seen.
- Bordt et al. only looked at classification, not at filling missing values or at MNAR.
- **So we still don't know how much of an LLM's cleaning skill is real understanding and how much is memory.**



Dataset

Two groups: famous datasets the LLMs have most likely seen, and a new one they couldn't have seen.

```
[2]: import pandas as pd
df = pd.read_csv("diabetes.csv")
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Figure 2: Pima Indians Diabetes, 768 rows (famous). The zeros in SkinThickness and Insulin are actually missing values. Source: kaggle.com/datasets/uciml/pima-indians-diabetes-database

```
[4]: diabd = pd.read_csv("DiaBD.csv")
diabd.head()
```

	age	gender	pulse_rate	systolic_bp	diastolic_bp	glucose	...	family_diabetes	hypertensive	family_hypertension	cardiovascular_disease	stroke	diabetic
0	42	Female	66	110	73	5.88	...	0	0	0	0	0	No
1	35	Female	60	125	68	5.71	...	0	0	0	0	0	No
2	62	Female	57	127	74	6.85	...	0	0	0	0	0	No
3	73	Male	55	193	112	6.28	...	0	0	0	0	0	No
4	68	Female	71	150	81	5.71	...	0	0	0	0	0	No

5 rows × 15 columns

Figure 3: DiaBD, 5,288 rows, released in 2025 (new). Source: Prama et al., Data in Brief, 2025; data.mendeley.com/datasets/m8cgwx9s6/2

Dataset (Contd.)

```
[3]: from sklearn.datasets import load_iris
iris = load_iris(as_frame=True).frame
iris.head()
```

[3]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Figure 4: Iris, 150 rows. Source: archive.ics.uci.edu/dataset/53/iris

```
[5]: from sklearn.datasets import load_wine
wine = load_wine(as_frame=True).frame
wine.head()
```

[5]:

	alcohol	malic_acid	ash	...	od280/od315_of_diluted_wines
0	14.23	1.71	2.43	...	3.92
1	13.20	1.78	2.14	...	3.40
2	13.16	2.36	2.67	...	3.17
3	14.37	1.95	2.50	...	3.45
4	13.24	2.59	2.87	...	2.93

5 rows × 14 columns

Figure 5: Wine, 178 rows. Source: archive.ics.uci.edu/dataset/109/wine

```
[7]: from sklearn.datasets import load_breast_cancer
bc = load_breast_cancer(as_frame=True).frame
bc.head()
```

[7]:

	mean radius	mean texture	mean perimeter	mean area	...	worst concave points	worst symmetry	worst fractal dimension	target
0	17.99	10.38	122.80	1001.0	...	0.2654	0.4601	0.11890	0
1	20.57	17.77	132.90	1326.0	...	0.1860	0.2750	0.08902	0
2	19.69	21.25	130.00	1203.0	...	0.2430	0.3613	0.08758	0
3	11.42	20.38	77.58	386.1	...	0.2575	0.6638	0.17300	0
4	20.29	14.34	135.10	1297.0	...	0.1625	0.2364	0.07678	0

5 rows × 31 columns

Figure 6: Breast Cancer Wisconsin (Diagnostic), 569 rows. Source: archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic



Dataset (Contd.)

Where the data comes from

Real

The original table, as published.

Perturbed

Same table with units changed, a little noise and rows shuffled, so it no longer matches word for word.

**Synthetic
twin**

Brand-new rows generated (SDV) to follow the same statistics. Nothing to remember.

What the columns are called

Real names

Glucose	BMI	Age
148	33.6	50

Hidden

f1	f2	f3
148	33.6	50

Shuffled

Age	Glucose	BMI
148	33.6	50

$3 \times 3 = 9$ **versions** of every dataset. We hide 10%, 20% or 30% of the values in **exactly the same cells** in all 9, so any change in accuracy comes from the version alone.

Proposed Algorithm

Every method gets the same puzzle. We change one thing at a time and watch whose accuracy drops.

1. **Collect datasets.** Famous ones (Iris, Wine, Breast Cancer, Pima) and a new one (DiaBD).
2. **Test memorisation.** Check how much of each dataset every LLM can recite, using tabmemcheck.
3. **Make 9 versions.** Real, perturbed and synthetic twin, each with real, hidden and shuffled column names.
4. **Hide values.** Remove 10–30% of cells (MCAR, MAR, MNAR, real Pima patterns), the same cells in all 9 versions.
5. **Fill the gaps.** Classical methods and local 4-bit LLMs, with zero-shot, few-shot and step-by-step prompts.
6. **Compare results.** Measure the error against the true values and work out the two effects.

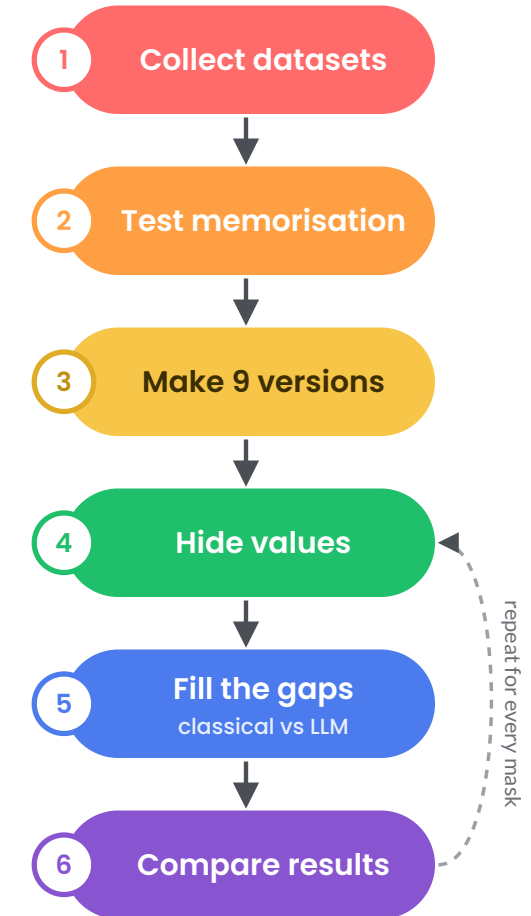


Figure 7: Steps at a glance

Proposed Algorithm (Contd.)

Algorithm 1 Benchmark procedure

```
1  Input: datasets  $D$ , classical methods  $C$ , LLMs  $M$ 
2  for each dataset  $d \in D$  do
3      score memorisation of  $d$  for each model in  $M$ 
4       $V \leftarrow 9$  versions of  $d$  (3 origins  $\times$  3 name styles)
5      for each mechanism  $g$ , rate  $r \in \{10, 20, 30\}\%$  do
6          mask  $\leftarrow$  hide cells using  $g, r$  // shared by all of  $V$ 
7          for each  $v \in V$ , method  $m \in C \cup M$  do
8              fill hidden cells of  $v$  with  $m$ 
9              record NRMSE, MAE, cost
10 compute  $\Delta_{\text{mem}}$ ,  $\Delta_{\text{sem}}$  and  $D_m$  using (1)–(3)
11 Output: effect sizes and method rankings
```

Key formulations

$$\Delta_{\text{mem}} = [E_{\text{LLM}}(\text{twin}) - E_{\text{LLM}}(\text{real})] - [E_{\text{CL}}(\text{twin}) - E_{\text{CL}}(\text{real})] \quad (1)$$

Extra error the LLM suffers when there is nothing to recall.

$$\Delta_{\text{sem}} = [E_{\text{LLM}}(\text{hidden}) - E_{\text{LLM}}(\text{names})] - [E_{\text{CL}}(\text{hidden}) - E_{\text{CL}}(\text{names})] \quad (2)$$

Extra error the LLM suffers when column names are hidden.

$$D_m = E_m(\text{MNAR}) - E_m(\text{MCAR}) \quad (3)$$

How much method m worsens under MNAR. Small on real data but large on the twin points to memorisation.

E = NRMSE of a method · CL = classical method · twin = synthetic copy

Proposed Architecture

- **Datasets** are loaded and kept complete, so the true values are always known.
- **Memorisation test** scores how well each LLM can recite each dataset.
- **Version maker** creates the 9 versions.
- **Mask generator** hides the same cells in every version.
- **Imputers**: classical methods and local LLMs fill the gaps. LLM answers go through a parser.
- **Evaluator** compares filled values with the true ones, and the analysis step works out the effects.

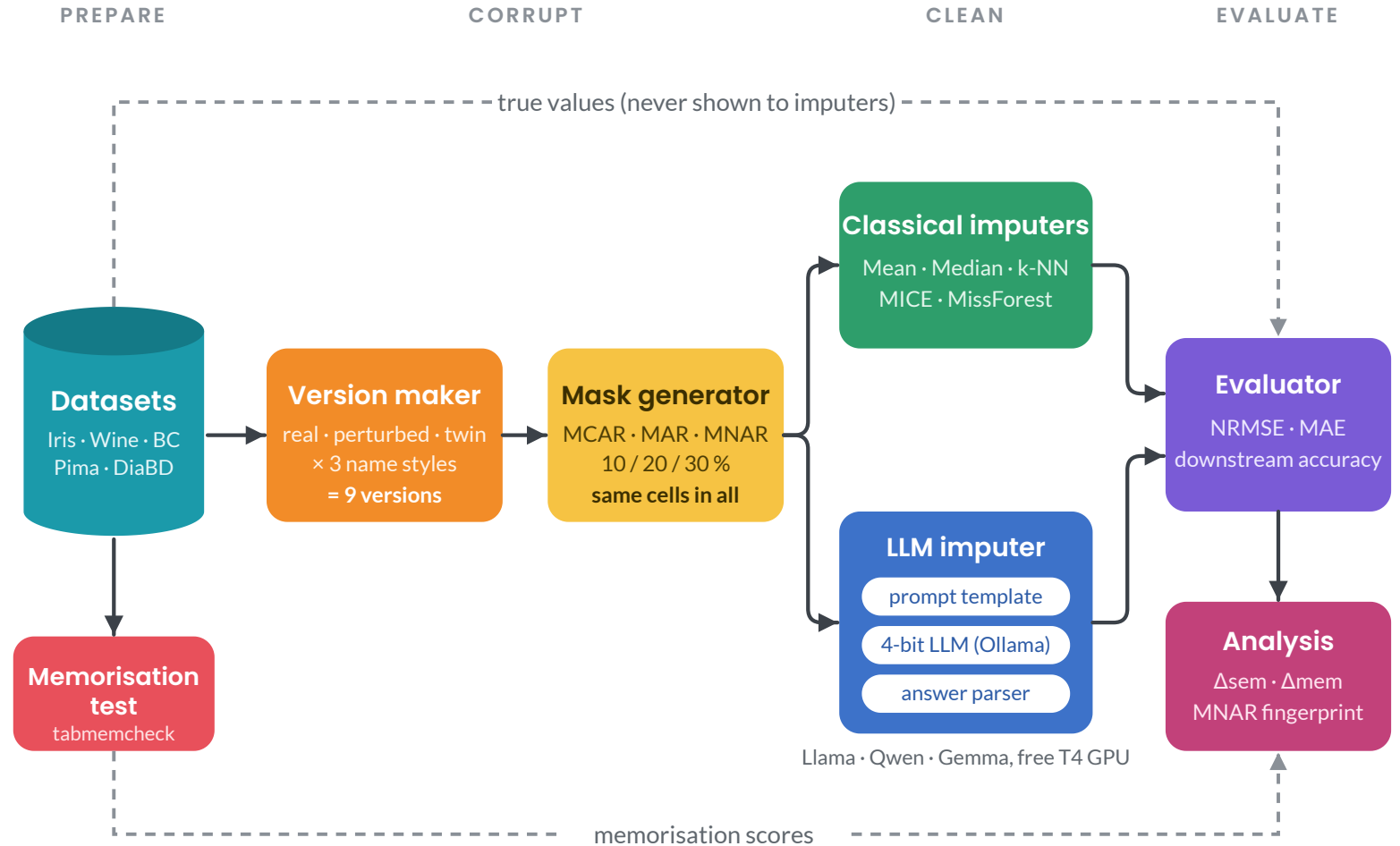


Figure 8: System architecture



Tools and Technologies

Language	Python 3.11
Classical imputers	scikit-learn, MissForest
Synthetic data	SDV (Gaussian Copula, CTGAN, TVAE)
LLMs	Ollama, 4-bit Llama 3.1, Qwen2.5, Gemma 2
Plots	Matplotlib, Seaborn

No paid API is used, so anyone can re-run the experiments.

Data handling	pandas, NumPy, DuckDB
Missing values	pyampute, own scripts
Memorisation test	tabmemcheck
Statistics	SciPy, statsmodels, XGBoost
Hardware	8 GB laptop, free Colab / Kaggle T4 GPU

Performance Evaluation

What we check	How
Accuracy of filled values	NRMSE* and MAE* for numbers, % correct for categories
Shape of the data	How close the filled values' distribution is to the real one
Usefulness	Accuracy of random forest, gradient boosting and logistic regression trained on the cleaned data
Cost	Model calls, tokens, time and invalid answers

Table 5: Evaluation metrics

$$\text{NRMSE} = \sqrt{\left(\frac{1}{n} \sum (\hat{x}_i - x_i)^2 \right) / s} \quad \text{MAE} = \frac{1}{n} \sum |\hat{x}_i - x_i|$$

* $\hat{x}_i$ = value filled in by the method, x_i = true value, n = number of hidden cells, s = spread of the column (range or standard deviation, to be fixed during implementation).

Performance Evaluation (Contd.)



How we'll test

- Baselines: mean, median, k-NN, MICE, MissForest
- 5 LLMs, each with 3 prompting styles (no examples, 4 examples, step by step)
- Every setting repeated with different random seeds
- Paired permutation tests with Holm–Bonferroni correction, Hedges' g and bootstrap confidence intervals

Work done so far

- ✓ Reviewed 6 recent papers and found the research gap
- ✓ Designed the benchmark: 9 versions, shared masks and the three formulas
- ✓ Designed the algorithm and system architecture
- ✓ Selected the datasets and verified their sources
- ✓ Chose free, local tools and models

Next steps

- Build the pipeline and run the memorisation tests
- Run the experiments; fix the train/test split and seeds
- Analyse the results and release the code

References



1. A. Hayat and M. R. Hasan, "A Context-Aware Approach for Enhancing Data Imputation with Pre-trained Language Models," COLING, pp. 5668–5685, 2025.
2. A. F. Abate, L. Cascone, V. Fraenza, M. Nappi and P. Ritrovato, "Constrained Conditional LLM-based Imputation via Iterative Chained Updates," Pattern Recognition Letters, 2026.
3. L. Li, L. Fang, B. Ludäscher and V. I. Torvik, "AutoDCWorkflow: LLM-based Data Cleaning Workflow Auto-Generation and Benchmark," Findings of EMNLP, pp. 7766–7780, 2025.
4. S. Bordt, H. Nori, V. Rodrigues, B. Nushi and R. Caruana, "Elephants Never Forget: Memorization and Learning of Tabular Data in Large Language Models," COLM, 2024.
5. X. He, Y. Ban, J. Zou, T. Wei, C. Cook and J. He, "LLM-Forest: Ensemble Learning of LLMs with Graph-Augmented Prompts for Data Imputation," Findings of ACL, pp. 6921–6936, 2025.
6. H. Zhang, Y. Dong, C. Xiao and M. Oyamada, "Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing," EMNLP, 2024.

References (Contd.)



7. J. W. Smith et al., "Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus," Proc. Symposium on Computer Applications and Medical Care, pp. 261–265, 1988. Pima Indians Diabetes Database, Kaggle.
8. T. T. Prama, M. J. Rahman, M. Zaman, F. Sarker and K. A. Mamun, "DiaBD: A Diabetes Dataset for Enhanced Risk Analysis and Research in Bangladesh," Data in Brief, vol. 61, 111746, 2025. Mendeley Data, V2.
9. R. A. Fisher, "Iris," UCI Machine Learning Repository, 1936.
10. S. Aeberhard and M. Forina, "Wine," UCI Machine Learning Repository, 1991.
11. W. Wolberg, O. Mangasarian, N. Street and W. Street, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1995.

Thank You

Questions are welcome

